Bachelor of Science in Information Technology

DATABASE SYSTEMS

Designing, Querying and Trusting a Relational Database

Unified Lecture Handout

Complete Course in One Volume • Fall 2026

A database is not a place to put data. It is a set of promises about data.

This course is how each promise is made — and, every time, what the database does when you do not ask it to keep one.

What this volume covers. Every clause of the HEC outline for *Database Systems*, traced chapter by chapter in the Course Specification overleaf.

That outline is published in the **2017** curriculum (p. 88) and reprinted *word for word* in the **2023** curriculum (p. 58) — the two differ by a single added full stop across ninety words, six years apart. The **2025** curriculum deletes it: it covers only BS Computer Science, publishes no course content for any major course at all, and supplies five rewritten learning outcomes with no Bloom levels. All of that is recorded in the Course Specification rather than smoothed over.

The course is taught on a **running database**. One schema is designed in Chapter 6, created in Chapter 10, and every chapter afterwards adds to it or asks something of it; by Chapter 23 it is the semester project. The tool is **SQLite**, which you already have.

Six Parts • Twenty-Three Chapters • Seven Appendices

How to Use This Handout

There are two ways to teach this subject badly.

The first is to teach it as theory. Relational algebra on a whiteboard, normal forms as definitions to be recited, ACID as four words beginning with the right letters. A student who has learned it this way can answer an examination question about third normal form and cannot look at a table and say whether it is in it.

The second is to teach it as a tool. Open the software, click here, type a SELECT, admire the rows. A student who has learned it this way can produce an answer and has no way of knowing whether it is the right one, because nothing in the course ever told them what the database was promising.

This handout does neither, and the reason is in the title. A database makes promises — that this attribute is a number, that this row refers to a row that exists, that either all of this happened or none of it did — and every one of those promises is *checkable*. So the method of this book is to state the promise, then ask the database to break it, and see what happens. Eleven chapters end with a figure produced that way on a real machine. They are marked, and the file that produces each one is named.

The furniture

Every chapter has the same parts, in the same order, so that you can find what you need without reading from the beginning.

You will see	It means
Learning Outcomes	What you should be able to *do* afterwards. Five per chapter, each with a verb you can be examined on
Before You Start	What this chapter assumes, and where it was taught
Key Terms	Every term defined here. If one is unfamiliar at the end, the chapter did not work
Definition	A term being given its precise meaning. These are the sentences to learn exactly
Concept	An idea, a rule of thumb, or a way of deciding
Watch Out	Something that is true, surprising, and will cost you if you do not know it
Example	A worked case, usually from the running database
Worked Example	Arithmetic, every step shown. Nothing is left "by inspection"
Measured	A figure produced by running the named file in `code/`, on the machine in Appendix A. You can reproduce every one
Common Pitfalls	What goes wrong, collected, so that you meet it here rather than in the laboratory
Try It Yourself	The laboratory. Numbered steps, run in order
Checkpoint	Three questions. Answers in Appendix C, and they are full answers rather than a line each
Chapter Summary	The chapter in eight or nine lines
Review Questions	Seven: two multiple choice, three short, two applied

The laboratory, and what it needs

Nothing to install. The whole course runs on **SQLite**, which is already on every machine you are likely to use: it ships inside Python, it ships with most operating systems, and a database is a single file you can copy, email or delete. Appendix A sets it up in one sitting, which for most readers means checking that it is already there.

That choice is not a compromise with the curriculum. **HEC names no database product in any edition** — the 2017 and 2023 outlines say only "in any DBMS", and 2025 says "popular database management systems". What SQLite can and cannot show you is stated plainly in Chapter 1 and again in Chapter **??**, because a tool's limits are part of what you are learning.

The database we build

One schema, carried through the whole course. You design it in Chapter **??** and Chapter **??**, map it to tables in Chapter **??**, create it in Chapter **??**, fill it, query it for five chapters, find out in Chapter **??** that it was wrong, fix it, index it, and defend it. It is a university enrolment database — students, courses, sections, instructors, enrolments, grades — chosen because you already know how it is supposed to behave, so no lecture time goes on explaining the subject matter.

If you read nothing else

- Chapter **??**, on the difference between *declaring* a key and *having* one.
- Chapter **??**, which is the chapter this course exists for.
- Chapter **??**, where the rule everybody repeats stops being true and you can see the exact row count at which it does.

Course Specification

This chapter exists so that a student, a lecturer or a reviewer can see exactly what this handout claims to cover and on what authority. Everything in it is traceable to a published document, and where those documents are wrong, unclear or contradict one another, that is recorded here rather than quietly corrected.

Particulars

Item	Value
Course title	*Database Systems*. HEC 2023's description page calls it *Database System*, singular, while every table in the same booklet says *Systems*. The plural is used here
Course code	**CS-103**. Unlike every sibling course in this series, there is no `INFT-` equivalent: the registrar uses CS-103 and nothing else
Programme	BS Information Technology, **2nd semester**. This matches HEC 2023, which places the course in semester 2 of every computing degree. HEC 2017 placed it at BSIT semester V; the discrepancy is recorded, not resolved
Credit hours	**4 (3+1)** — three lecture credits and **one credited laboratory**. The laboratory is paid for, which is why every chapter has one and why they are long enough to be worth the hour
Prerequisite	**None.** HEC 2017's own booklet gives two different answers — see defect 4
Duration	Sixteen teaching weeks, midterm in Week 8, project due Week 16 (Appendix G)
Teaching methodology	As published (HEC 2017): lectures, written assignments, **practical labs**, semester project, presentations
Assessment	As published (HEC 2017): sessional exam, home assignments, quizzes, project, presentations, final exam
Set texts	As published: Connolly and Begg; Garcia-Molina, Ullman and Widom; Silberschatz, Korth and Sudarshan; Ramakrishnan and Gehrke. No publication years are given for any of the four, in any edition — see Appendix E

Where the outline comes from

Edition	Status	Semester	Course content published?
2017	Computing Core — compulsory in BSCS, BSSE *and* BSIT	IV, IV, **V**	**Yes** — p. 88

Edition	Status	Semester	Course content published?
2023	Computing Core — compulsory in all ten computing degrees and the Associate Degree	**2**	**Yes** — p. 58, *the 2017 text reprinted*
2025	Major Compulsory — but the edition covers only BS Computer Science and the Associate Degree. **There is no BSIT in it at all**	III	**No** — deleted; five CLO bullets only

⚠ Six defects in the published documents

Recorded rather than inherited. Each is stated with the correction this handout uses.

1. **The syllabus is frozen.** The 2023 course outline is the 2017 course outline word for word, with **one added full stop** ("vs" becomes "vs.") across ninety words — six years apart. Then 2025 removes it entirely. A field that changed a great deal in that period is described by a text that did not change at all.
2. **The outline normalises before it designs.** The clause list reaches *normalization, functional dependencies, normal forms* **before** *entity relationship model . . . entity-relationship diagrams*. You cannot normalise a schema you have not designed. All four of HEC's own reference books teach the other order. **This handout reorders**: design in Part II, map in Part III, normalise in Part V. The clause numbers in the table below show exactly where each one moved to.
3. **One clause is printed twice.** "attributes" appears in the relational-model run and again in the entity-relationship run. Thirty three clauses, thirty two distinct. Both occurrences are traced below and the second is marked as a duplicate.
4. **The prerequisite contradicts itself inside one booklet.** HEC 2017's course description page says `Prerequisites: None`. All three of its scheme-of-studies tables — BSCS p. 25, BSIT p. 31, BSSE p. 39 — say **Data Structures & Algorithms**. At 2nd semester the department cannot assume Data Structures, so **this handout assumes none**, and Chapter **??** and Chapter **??** are written to be self-contained because of it.
5. **The 2023 learning-outcome table is broken in print.** CLO-3's text stops mid-sentence; its remainder ("anomalies by normalizing database tables.") is printed as CLO-4's text; and the real CLO-4 is left in a trailing row with *no number and no Bloom level*. This is a typesetting fault in the published national curriculum, not a misreading. The outcomes are reconstructed below from the 2017 text, which is intact.
6. **A Bloom level is wrong.** CLO-3 reads "Identify functional dependencies and *resolve* database anomalies by normalizing database tables" and is tagged **C2**. Identifying is C2; resolving and normalising are C3 at least. Every sibling course in this series complains that HEC left the Bloom column blank. Here it is filled in — and filled in incorrectly, which is harder to notice.

Course content, traced clause by clause

The outline is reproduced below exactly as published, in its published order, split into its thirty-three clauses. The right-hand column is where each is taught. Clauses that move are the subject of defect 2.

#	Clause, as published	Taught in
1	Basic database concepts	Chapter 1
2	Database approach vs file based system	Chapter 1
3	database architecture	Chapter 3
4	three level schema architecture	Chapter 2
5	data independence	Chapter 2
6	relational data model	Chapter ??
7	attributes	Chapter ??
8	schemas	Chapter ??
9	tuples	Chapter ??
10	domains	Chapter ??
11	relation instances	Chapter ??
12	keys of relations	Chapter ??
13	integrity constraints	Chapter ??
14	relational algebra	Chapter ??
15	selection	Chapter ??
16	projection	Chapter ??
17	Cartesian product	Chapter ??
18	types of joins	Chapter ??
19	**normalization**	Chapter ?? *(moved later)*
20	**functional dependencies**	Chapter ?? *(moved later)*
21	**normal forms**	Chapter ?? *(moved later)*
22	**entity relationship model**	Chapter ?? *(moved earlier)*
23	entity sets	Chapter ??
24	attributes	**duplicate of 7**
25	relationship	Chapter ??, Chapter ??
26	entity-relationship diagrams	Chapter ??, Chapter ??
27	Structured Query Language (SQL)	Chapter ??, Chapter ??
28	Joins and sub-queries in SQL	Chapter ??, Chapter ??
29	Grouping and aggregation in SQL	Chapter ??
30	concurrency control	Chapter ??
31	database backup and recovery	Chapter ??
32	indexes	Chapter ??
33	NoSQL systems	Chapter ??

Three topics taught that the outline does not name

A reviewer will ask, so the justification is here rather than implied.

- **Chapter ??, changing a schema that is already full.** The outline never mentions ALTER. A student who cannot change a schema cannot carry one database through sixteen weeks, and the project requires exactly that.
- **Chapter ??, views and triggers.** Absent from the clause list. Required by CLO-2, which asks for *logical and physical* schemas, and assumed by the department's own *E-Commerce Application Development* (INFT-4041), which lists this course as its prerequisite — see Appendix F.
- **Chapter ??, security and authorisation.** Absent from 2017 and 2023 entirely. **HEC 2025 requires it by name**: its fifth learning outcome is "Analyze the role of databases in information systems and their security considerations". The newest edition justifies the addition.

Course Learning Outcomes

HEC 2017's four, verbatim, with the Domain and Bloom levels exactly as published — including the one that is wrong.

CLO	As published	Level	Taught in
CLO-1	"Explain fundamental database concepts."	C2	Part I
CLO-2	"Design conceptual, logical and physical database schemas using different data models."	C5	Parts II, III
CLO-3	"Identify functional dependencies and resolve database anomalies by normalizing database tables."	**C2** †	Part V
CLO-4	"Use Structured Query Language (SQL) for database definition and manipulation in any DBMS."	C4	Parts III, IV

† **As published, and incorrect** — see defect 6. Assessed in this course at **C3**.

Four added by the department, each discharging one of HEC 2025's five outcomes, which the 2017 clause list does not reach.

CLO	Level	Outcome	Discharges
CLO-5	C3	Write SQL for retrieval, insertion, update and deletion against a schema you did not design	HEC 2025 outcome 2
CLO-6	C2	Explain transactions, concurrency and recovery, and demonstrate what a transaction prevents	HEC 2025 outcome 3
CLO-7	C3	Implement and administer a working database, including backup and restore	HEC 2025 outcome 4
CLO-8	C4	Analyse a database's role in a wider system and its security posture, and justify the analysis from evidence	HEC 2025 outcome 5

Prerequisite Self-Check

This is a **second-semester** course. It assumes one semester of programming and nothing else — in particular it does *not* assume Data Structures, whatever HEC 2017's scheme-of-studies tables say (defect 4).

Twelve questions. If you can answer ten, start at Chapter 1. If you cannot, the right-hand column says where to look — and none of it will take more than an evening.

#	Question	If not
1	What is the difference between a file and a folder?	—
2	A text file holds one record per line, fields separated by commas. How would you find every line whose third field is "2026"?	Programming Fundamentals
3	Write a loop that reads a file line by line.	Programming Fundamentals
4	What does it mean for two values to be *equal* when one is the number 7 and the other is the text "7"?	Chapter **??** returns to this
5	What is an array index? What happens if you use one that is too large?	Programming Fundamentals
6	Given sets $A = \{1, 2, 3\}$ and $B = \{2, 3, 4\}$, write $A \cup B$, $A \cap B$ and $A - B$.	Appendix B
7	What does $A \times B$ mean for two sets, and how many elements does it have?	Appendix B
8	Is the statement "$x > 5$ and $x < 3$" ever true?	Appendix B
9	What is the truth value of "not (P and Q)" when P is true and Q is false?	Appendix B
10	Two people edit the same file at the same time and both save. What happens to the first person's changes?	Chapter **??**
11	Roughly how many times do you have to halve 1,000,000 to get to 1?	Appendix B; Chapter **??** needs it
12	What is a function, in the mathematical sense — how many outputs may one input have?	Appendix B; Chapter **??** depends on it

Question 12 is the one that matters

Everything in Part V rests on it. A *functional dependency* is nothing more than the statement that one set of attributes is a function of another: give me a student's roll number and there is exactly one name it can be paired with. If question 12 was uncomfortable, read Appendix B before Chapter **??**, not during it.

Contents

How to Use This Handout ii

Course Specification iv

Prerequisite Self-Check viii

I What a Database Is 1

1 Why Not Just Files? 3
1.1 The Program You Have Already Written . . . 3
1.2 Six Things That Go Wrong . . . 4
1.3 What a Database Management System Is . . . 5
1.4 When a File Is Still the Right Answer . . . 6

2 Three Levels, and Why They Are Separate 9
2.1 One Database, Three Descriptions . . . 9
2.2 Schema and Instance . . . 10
2.3 Data Independence, Which Is the Point of All This . . . 11
2.4 Where SQLite's Three Levels Are . . . 11

3 What Is Inside a DBMS, and Who Does What 15
3.1 A DBMS Is Not One Program . . . 15
3.2 Following One Statement Down . . . 16
3.3 The Data Dictionary Is Just More Tables . . . 17
3.4 The Four Roles . . . 17

4 Your First Database, in One Command 21
4.1 Three Statements and a File . . . 21
4.2 Writing the Schema Down Before You Type It . . . 22
4.3 What It Actually Cost . . . 22
4.4 The Eight Dot-Commands . . . 24

Appendices 28

A The Toolkit: Setting It Up and Keeping It 29
A.1 What You Need . . . 29
A.2 Why SQLite, and What It Costs You . . . 29
A.3 The Ten Commands You Will Actually Use . . . 30
A.4 Resetting to a Known State . . . 31
A.5 The Machine Every Timing Was Taken On . . . 31
A.6 Scope: What You May Point Any of This At . . . 31

B The Sixteen-Week Schedule **33**
B.1 The Schedule 33
B.2 How the Assessment Adds Up 34

List of Figures

P1.1 Reading path through Part I. Chapters 1 to 3 are the argument; Chapter 4 is the first one you can run, and by the end of it there is a file on your disk that every later chapter adds to. 2

1.1 The same three programs, two ways. The difference is not that the right-hand side has fewer files — it is that the rules about what a student is have moved out of the programs and into one place. 4

2.1 The ANSI/SPARC three-level architecture. The two mappings are the whole value of the arrangement: because they exist, a change at one level can be absorbed instead of propagated. 10

3.1 The components of a DBMS, and the path a statement takes down through them. The optimiser and the parser both consult the data dictionary — the parser to check that `student` exists and has a `name`, the optimiser to find out how big it is and what indexes it has. 16

4.1 One table, with Chapter 2's schema/instance distinction drawn on it. The box above the rule is the schema and is four lines long whether the table holds one row or a million. 22

Part I

What a Database Is

Every program you have written so far has stored its data in a file, and that worked. Part I is about the point at which it stops working — not in theory, but on a particular afternoon when two people save at once, or a column turns out to mean two different things, or somebody deletes a row that something else was relying on.

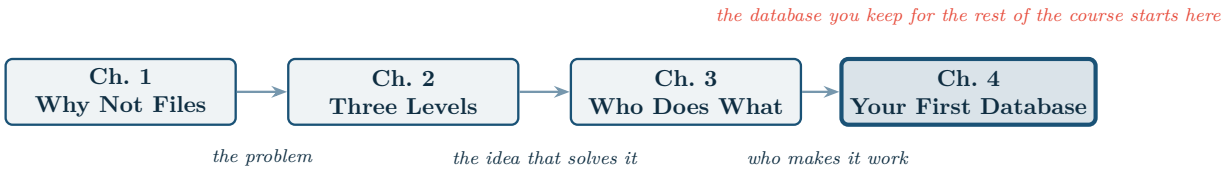


Figure P1.1: Reading path through Part I. Chapters 1 to 3 are the argument; Chapter 4 is the first one you can run, and by the end of it there is a file on your disk that every later chapter adds to.

Chapter 1

Why Not Just Files?

Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** how a file-based application stores and finds data, and **state** what it costs to add one field.
- **Name** the six problems a file-based approach has, and **give** a concrete example of each.
- **Explain** what a database and a database management system are, and **distinguish** the two.
- **Judge** when a file is the right answer, which is more often than this chapter's title suggests.
- **Predict** which of the six problems a given scenario will hit first.

Before You Start

One semester of programming: reading and writing a file, loops, and what a record looks like when it is a line of comma-separated text. Nothing else. Appendix A if you have not checked that SQLite is on your machine.

Key Terms in This Chapter

data • information • file-based system • database • database management system (DBMS) • data redundancy • data inconsistency • data isolation • integrity constraint • atomicity • concurrent access • lost update • data security • application program

1.1 The Program You Have Already Written

Somewhere in your first semester you wrote a program that kept a list of things in a file. It probably looked like this, and there is nothing wrong with it:

```
# students.csv
# 2021-IT-001,Ayesha Khan,BSIT,3
# 2021-IT-002,Bilal Ahmed,BSIT,3

def find(roll_no):
    for line in open("students.csv"):
        fields = line.strip().split(",")
        if fields[0] == roll_no:
            return fields
    return None
```

It works. It is twelve lines. For a list of your own books, or a configuration file, or the output of a program that nobody else reads, it is the right answer and this chapter is not an argument against it.

This chapter is about the afternoon it stops being the right answer — and about the fact that it does not stop gradually. A file-based approach works, and works, and works, and then the department asks for one more thing and suddenly nothing works.

Data, Information, and the difference

Data are raw recorded facts: the characters `2021-IT-001` in a file.

Information is data in a context that makes it answer a question: *the roll number of the student who scored highest in Database Systems last term.*

A **file-based system** stores data and leaves every question to the application program. A **database** stores data *together with a description of what it means and what must be true of it*, so that some of the questions — and all of the guarantees — can be answered by the store itself.

That last sentence is the whole subject. Everything in the remaining twenty-two chapters is a particular case of it.

1.2 Six Things That Go Wrong

These are not six opinions. Each is a specific failure with a name, and you will meet all six in a working life.

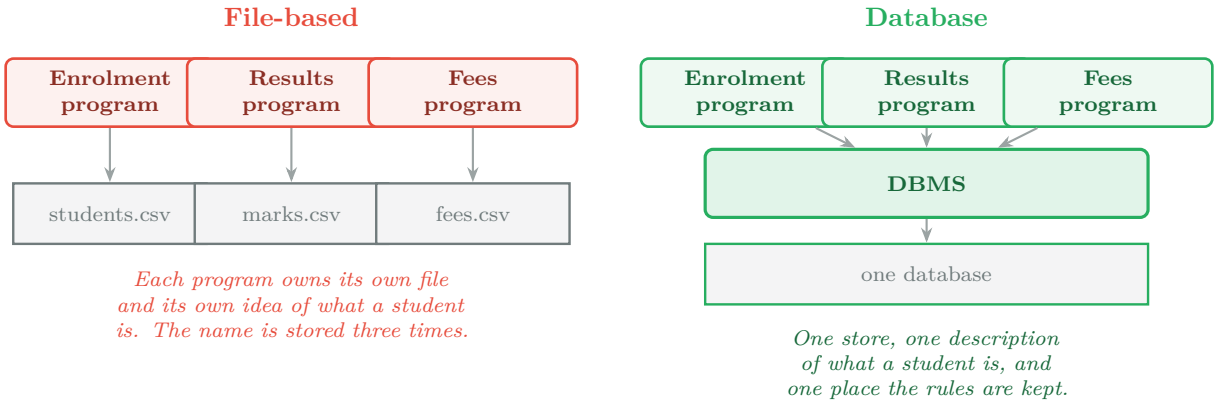


Figure 1.1: The same three programs, two ways. The difference is not that the right-hand side has fewer files — it is that the rules about what a student is have moved out of the programs and into one place.

The six, with a case for each

1. **Data redundancy.** The student's name is in `students.csv`, in `marks.csv` so the results sheet can print it, and in `fees.csv` so the receipt can. Three copies, and disk space is the least of it.
2. **Data inconsistency.** A student marries and changes her name. Two of the three files are updated. The college now holds two different answers to one question, and *nothing in the system is wrong* — each file is internally fine. This is redundancy's actual cost.
3. **Data isolation.** The question "which students owe fees and are enrolled in a course that has already started?" needs two files in different formats. There is no way to ask it; there is only a program somebody has to write.
4. **Integrity problems.** "A student's semester is between 1 and 8." Where does that rule live? In the enrolment program. And in the results program, where somebody wrote 1 to 10. The rule is a comment in three places and enforced in none.
5. **Atomicity problems.** Transferring a student between sections means removing a line from one file and adding it to another. The machine loses power in between. The student is now in no section at all, and nothing can tell you that happened.

6. **Concurrent access and security.** Two clerks open the file, both edit, both save; the second overwrites the first and neither is told — a **lost update**. And the fees clerk, who needs one column, has the whole file including every home address.

Which one bites first, and why it is never the one you expect

Students asked to rank these almost always put redundancy first, because it is the one that is visible — you can *see* the duplicated column.

In practice the order is almost exactly reversed. Redundancy costs disk, which is cheap. **Inconsistency** is what gets noticed, because somebody receives a letter addressed to a name they no longer use. **Isolation** is what gets the department to pay for a new system, because the question they want answered cannot be answered at all. And **atomicity** is the one nobody finds until the day it matters, because a half-finished transfer looks exactly like a student who was never enrolled.

Keep this ranking in mind in Chapter **??**, when you have to justify your design to somebody who will ask what problem it solves.

1.3 What a Database Management System Is

Database, DBMS, Database System

A **database** is a shared, organised collection of data, together with a description of that data — what it is called, what type it is, what must be true of it, and how its parts relate.

A **database management system** (DBMS) is the *software* that creates, maintains and controls access to a database. SQLite is a DBMS; so are Oracle, PostgreSQL, MySQL and SQL Server.

A **database system** is the whole arrangement: the data, the DBMS, the application programs that use it, and the people.

These three are routinely used as if they meant the same thing, including by people who should know better. In an examination they do not, and the distinction is CLO-1's business.

The DBMS is what the right-hand side of Figure 1.1 adds. It is a single piece of software that every program goes through, and because everything goes through it, it is the one place where a rule can be stated once and be true for everybody.

What moves into the DBMS, problem by problem

Redundancy	One copy of each fact, and other tables refer to it (Chapter **??**)
Inconsistency	Follows from the above — there is nothing to disagree with
Isolation	One query language over all the data (Chapter **??** onward)
Integrity	Constraints declared once, in the schema (Chapter **??**)
Atomicity	Transactions: all of it or none of it (Chapter **??**)
Concurrency, security	Locking, and authorisation (Chapter **??**, Chapter **??**)

That table is this book's table of contents, rearranged. Every chapter from here is one row of it, done properly.

1.4 When a File Is Still the Right Answer

A course called Database Systems has an obvious bias, and you should know about it.

Use a file when every one of these is true

- **One program reads it, and one writes it.** No concurrency, so no lost updates.
- **It is small enough to read entirely.** If you always want all of it, an index buys nothing.
- **The questions never change.** Isolation only hurts when somebody asks something new.
- **Losing it is survivable**, or it can be regenerated.

Configuration files, log files, a cache, the output of a nightly job — all files, all correctly so. **The failure is not using a file; it is using a file after the fourth condition stopped holding and not noticing.**

Common Pitfalls

- **Treating redundancy as the main problem.** It is the cheapest of the six and the easiest to see, which is why it gets the attention.
- **Saying "database" when you mean "DBMS".** The data is not the software.
- **Believing a DBMS removes the need to think about integrity.** It gives you somewhere to put the rules. It does not write them, and Chapter **??** measures what happens when you declare one and it is not enforced.
- **Assuming a database is always better.** Four conditions above say otherwise.
- **Calling a spreadsheet a database.** It has rows and columns and none of the six guarantees.

Try It Yourself: Break a file-based system on purpose

No database yet. This laboratory is about earning the next twenty-two chapters.

```
# --- 1. Make the files -------------------------------------------------
# Three CSV files, by hand, in a text editor. Five students each.
#   students.csv   roll_no,name,programme,semester
#   marks.csv      roll_no,name,course,marks
#   fees.csv       roll_no,name,amount,paid

# --- 2. Count the redundancy -------------------------------------------
# How many times does one student's NAME appear across the three files?
# Multiply by 4,000 students. Write the number down.

# --- 3. Create an inconsistency ----------------------------------------
# Change one student's name in students.csv and marks.csv but NOT in
# fees.csv. Now answer: what is that student's name? Justify it.

# --- 4. Hit data isolation ---------------------------------------------
# Write the program that answers: "which students owe fees AND are
# enrolled in a course where they scored below 50?"
# Time yourself. Do not finish it if it takes more than twenty minutes
# -- stopping is the result.

# --- 5. Break atomicity ------------------------------------------------
# Write a script that moves a student from one file to another, with a
# deliberate crash between the two writes. Run it. Describe the state.

# --- 6. Simulate a lost update -----------------------------------------
```

```
# Open students.csv in two editors. Change a different row in each.
# Save both. Report what survived.

# --- 7. Write the memo ------------------------------------------------
# One page to a head of department who has asked why the existing
# system should be replaced. Use YOUR numbers from steps 2 and 4.
# No jargon: the reader has not taken this course.
```

What to notice. Step 4 is the one that convinces people. Redundancy is an argument; a question that cannot be answered at all is a business problem, and it is the one that gets a project funded.

Checkpoint — Test Yourself

1. A college keeps student records in one file and library loans in another, each maintained by one clerk who never touches the other. Which of the six problems does this arrangement have, and which does it genuinely escape? Say what would have to change for your answer to change.
2. Distinguish a database from a database management system, and explain why "we bought a database" is a sentence that needs clarifying before you can act on it.
3. Your colleague argues that redundancy is harmless because disk is cheap. Give the argument you would make in reply, name the problem redundancy actually causes, and give an example in which no single file is wrong and the system still is.

Chapter Summary

- Data are recorded facts; information is data in a context that answers a question. A database stores data *with a description of what it means and what must be true of it*, which is the whole subject.
- A file-based system works until it does not, and the change is abrupt rather than gradual.
- The six problems are redundancy, inconsistency, isolation, integrity, atomicity, and concurrent access with security.
- Redundancy is the visible one and the cheapest. Inconsistency is what gets noticed, isolation is what gets a project funded, and atomicity is the one nobody finds until it matters.
- A *database* is the data; a *DBMS* is the software; a *database system* is the whole arrangement including the people.
- Everything goes through the DBMS, which is why it is the one place a rule can be stated once and be true for everybody.
- A file is still right when there is one reader, one writer, a fixed set of questions and nothing irreplaceable. The failure is not noticing when that stops being true.
- The table in Section 1.3 is this book's contents rearranged: each of the six problems is solved by a named later chapter.

Review Questions

1. (MCQ) Two files hold a student's name and only one is updated. This is an example of: (a) redundancy; (b) inconsistency; (c) isolation; (d) an integrity constraint.

2. (MCQ) The software that creates and controls access to a database is called: (a) the database; (b) the database system; (c) the DBMS; (d) the schema.

3. (Short) Give a question about a university that cannot be answered from two separate CSV files, and say which of the six problems that is.

4. **(Short)** Explain atomicity using an example that is not a bank transfer.

5. **(Short)** State two conditions under which a plain file is a better choice than a database, and give a real example of each.

6. **(Applied)** A clinic keeps appointments in a spreadsheet shared on a network drive, edited by three receptionists. Identify every one of the six problems it has, rank them by which will cause a complaint first, and justify your ranking.

7. **(Applied)** You are asked to justify replacing a working file-based system. Write the three strongest arguments in order, say what evidence you would gather for each before the meeting, and state which argument you would drop if you only had five minutes.

Chapter 2

Three Levels, and Why They Are Separate

Learning Outcomes

By the end of this chapter you will be able to:

- **Name** the three levels of the ANSI/SPARC architecture and **state** what each one describes.
- **Distinguish** a schema from an instance, and **explain** why the difference matters when something changes.
- **Define** logical and physical data independence, and **give** an example of a change that each one protects you from.
- **Identify** which level a given change belongs to.
- **Explain** why a system with no separation of levels is the file-based system of Chapter 1 again.

Before You Start

Chapter 1: the six problems, and what a DBMS is. Nothing else. This chapter is the one piece of pure architecture in the course and it is here because every later chapter quietly assumes it.

Key Terms in This Chapter

ANSI/SPARC architecture • external level • external schema • user view • conceptual level • conceptual schema • internal level • internal schema • physical level • schema • instance • mapping • logical data independence • physical data independence

2.1 One Database, Three Descriptions

In Chapter 1 the fees program and the results program each had their own file and their own idea of what a student was. Putting them behind one DBMS fixes the duplication — but it creates a new problem immediately.

The fees clerk wants a student to be a roll number, a name and an amount owing. The examinations officer wants a roll number, a name and a list of marks. The person who has to make the database fast wants to know that the student table is sorted by roll number and has an index on surname. All three are talking about the same data and none of them should have to care about the other two.

The answer, standardised in 1975 and unchanged since, is to describe the database *three times*.

The three levels

The **external level** is what a particular user or program sees: their own tailored view of the part of the database they need, in the shape they need it. There are many external schemas, one per group of users.

The **conceptual level** is the whole database, described once, logically: every entity, every attribute, every relationship and every constraint — and nothing at all about how it is stored. There is exactly one conceptual schema.

The **internal level** is how it is actually stored: files, pages, record layouts, indexes, compression. There is exactly one internal schema.

A **mapping** connects adjacent levels, so that the DBMS can translate a request expressed against one into an operation on the next.

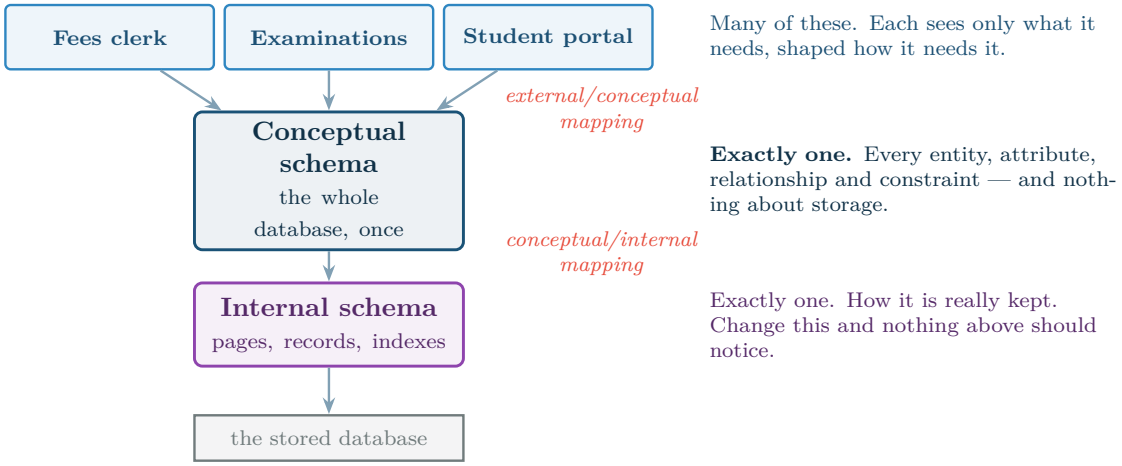


Figure 2.1: The ANSI/SPARC three-level architecture. The two mappings are the whole value of the arrangement: because they exist, a change at one level can be absorbed instead of propagated.

2.2 Schema and Instance

Two words that students use interchangeably for about a fortnight, and then cannot afford to.

Schema, Instance

A **schema** is the *description*: the names of the tables, their columns, the types, the constraints. It is set when the database is designed and changes rarely.

An **instance** (or *state*, or *extension*) is the data *in* the database at one moment. It changes every time anybody inserts a row.

The analogy worth keeping: a schema is a variable's declaration, an instance is its value. `int count;` is the schema; `7` is the instance.

The same distinction, three ways

	Schema	Instance
Programming	`int count;`	7
A database	STUDENT(roll_no, name, programme)	the 4,000 rows in it today
A form	the blank form	one filled-in copy

When somebody says "the database changed", the first question is always which of the two they mean, because the answers are days apart in effort.

2.3 Data Independence, Which Is the Point of All This

Three descriptions and two mappings are a lot of machinery. It buys exactly one thing, and the thing is worth it.

Logical and physical data independence

Physical data independence is the ability to change the *internal* schema — how data is stored — without changing the conceptual schema or any application program. Adding an index, moving to a different disk, reorganising the file: none of these should require a line of application code to change.

Logical data independence is the ability to change the *conceptual* schema — adding a table, adding a column, splitting one table into two — without changing the external schemas or the programs that use them.

Physical independence is easy and essentially achieved. **Logical independence is hard and only partly achieved**, which is why it is worth understanding where it fails.

Which level does this change belong to?

A question worth being able to answer instantly, because the cost of a change is decided by its level.

- **"Add an index on surname."** Internal. Nothing else changes. Chapter **??** does exactly this and measures it.
- **"Add a telephone number to students."** Conceptual. Existing views that never mentioned it carry on working — this is logical independence succeeding.
- **"Split the student table into student and student-contact."** Conceptual, and this is where logical independence is *tested*: Chapter **??** shows how a view can hide the split from programs that still want one table. Chapter **??** does this split for real.
- **"The fees clerk should not see home addresses."** External. A view, and in a server DBMS a privilege (Chapter **??**).
- **"Store dates as text rather than numbers."** Looks internal. It is *conceptual*, because it changes what comparisons mean — and Chapter **??** measures what SQLite does about that.

The last one is the one that catches people, and it catches them because "how it is stored" sounds internal in English and is not.

2.4 Where SQLite's Three Levels Are

An architecture standardised in 1975 is worth nothing to you unless you can point at its three levels in the tool on your own machine, which at first glance appears to have only one.

All three are there; only two can be enforced

It is worth knowing how an architecture from 1975 maps onto the tool in front of you, because at first glance SQLite seems to have only one level.

- **Conceptual:** your CREATE TABLE statements. Exactly one set of them, and `.schema` prints it.
- **Internal:** the page structure, the B-trees, the indexes. Chapter 4 measures it directly and you never write it — SQLite decides, and PRAGMA is the only window onto it.

- **External:** views (Chapter **??**). SQLite has them, so the level exists — but it has no users, so it cannot restrict anybody *to* a view. The level is present and the enforcement is not, which is Chapter **??**'s subject.

This is a real limitation and worth stating now rather than in week sixteen. You will build all three levels in this course. You will only be able to *enforce* two of them.

Common Pitfalls

- **Confusing schema with instance.** A schema change is a project; an instance change is a Tuesday.
- **Thinking there are many conceptual schemas.** There is one. Many external, one conceptual, one internal.
- **Believing physical independence means performance does not matter.** It means you can change storage *without rewriting programs*. It does not mean the choice is free — Chapter **??** measures exactly how unfree it is.
- **Calling a storage-format change internal when it changes meaning.** If a comparison starts giving different answers, it was conceptual.
- **Assuming logical independence is automatic.** It is the hard one, and Chapter **??** is how it is bought.

Try It Yourself: Find the three levels in a database you did not design

```
# --- 1. Make a database with something in it -------------------------
sqlite3 arch.db
```

then, in the shell:

```
-- --- 2. State a conceptual schema -------------------------------
CREATE TABLE student (
  roll_no   TEXT PRIMARY KEY,
  name      TEXT NOT NULL,
  programme TEXT NOT NULL,
  semester  INTEGER NOT NULL
);
INSERT INTO student VALUES
  ('2021-IT-001','Ayesha Khan','BSIT',2),
  ('2021-IT-002','Bilal Ahmed','BSIT',2),
  ('2021-CS-007','Hira Nadeem','BSCS',4);

-- --- 3. Read the conceptual schema back -------------------------
.schema student
-- That text IS the conceptual schema. Nothing in it mentions storage.

-- --- 4. Build an external schema --------------------------------
CREATE VIEW fees_view AS
  SELECT roll_no, name FROM student;
SELECT * FROM fees_view;
-- A user given only this cannot see "programme" or "semester".

-- --- 5. Look at the internal schema -----------------------------
PRAGMA page_size;
PRAGMA page_count;
SELECT name, type FROM sqlite_master;
-- You did not write any of this and cannot usefully change it.

-- --- 6. Change the internal level and prove nothing else moved --
CREATE INDEX idx_name ON student(name);
```

```
SELECT * FROM fees_view;
-- Same answer. That is physical data independence, demonstrated.

-- --- 7. Now change the conceptual level -------------------------
ALTER TABLE student ADD COLUMN email TEXT;
SELECT * FROM fees_view;
-- Still works, and still has two columns. Logical independence.

-- --- 8. Break it ------------------------------------------------
-- Try to drop the column the view depends on:
ALTER TABLE student DROP COLUMN name;
-- Write down exactly what happens, and which level failed.
```

What to notice. Steps 6 and 7 are the two kinds of independence, each demonstrated in one line. Step 8 is the limit of the second one, and the error message is worth reading twice.

Checkpoint — Test Yourself

1. Classify each of these by level, and say which kind of data independence (if any) protects the programs from it: adding an index; adding a column; renaming a column; storing the database on a faster disk; giving the library staff a view with no home addresses.
2. Explain the difference between a schema and an instance to somebody who has written programs but never used a database, using an example that is not a database.
3. SQLite has views but no users. Say precisely which of the three levels is affected by that, what you can still do, what you cannot, and how you would achieve the same effect in an application built on SQLite.

Chapter Summary

- The ANSI/SPARC architecture describes one database three times: *many* external schemas, *one* conceptual schema, *one* internal schema, joined by two mappings.
- The external level is what a user sees; the conceptual level is the whole database logically and says nothing about storage; the internal level is how it is really kept.
- A *schema* is the description and changes rarely; an *instance* is the data and changes constantly. A schema is a declaration, an instance is a value.
- **Physical data independence** lets the internal schema change without touching programs. It is essentially achieved.
- **Logical data independence** lets the conceptual schema change without touching external schemas. It is hard, partial, and bought with views (Chapter **??**).
- The level a change belongs to decides what it costs. "Store dates as text" sounds internal and is conceptual, because it changes what comparisons mean.
- In SQLite the conceptual level is your CREATE TABLE statements, the internal level is pages and B-trees you never write, and the external level is views — which exist but cannot be enforced, because there are no users to restrict.

Review Questions

1. (MCQ) How many conceptual schemas does a database have? (a) one per user; (b) one per table; (c) exactly one; (d) one per application.

2. (MCQ) Adding an index without changing any program demonstrates: (a) logical data independence; (b) physical data independence; (c) an external schema; (d) an instance change.

3. (Short) Give one change that logical data independence protects you from and one that it does not, and say what makes the difference.

4. **(Short)** Why does the architecture need *mappings* as well as schemas? What would be lost without them?

5. **(Short)** A colleague says the internal level does not matter because of physical data independence. Reply in two sentences.

6. **(Applied)** A university wants three groups to use one student database: admissions, examinations, and the students themselves. Design the external schemas — what each sees and what each must not — and say which parts of your design SQLite can enforce and which it cannot.

7. **(Applied)** Your conceptual schema must change: the single `address` column becomes four columns. Describe how you would make that change without breaking existing programs, what the view would look like, what it would cost, and at what point you would decide the view is no longer worth keeping.

Chapter 3

What Is Inside a DBMS, and Who Does What

Learning Outcomes

By the end of this chapter you will be able to:

- **Name** the major components of a DBMS and **state** what each one is responsible for.
- **Trace** a single SELECT statement through those components, in order, from text to answer.
- **Explain** what the data dictionary is, and **query** it.
- **Distinguish** the four database roles — administrator, designer, application programmer, end user — by what each one decides.
- **Identify** which components SQLite has, which it lacks, and which role has to cover for the missing ones.

Before You Start

Chapter 2: the three levels and the two mappings. This chapter is largely about *who performs* those mappings.

Key Terms in This Chapter

query processor • parser • query optimiser • execution engine • storage manager • buffer manager • file manager • transaction manager • lock manager • recovery manager • authorisation manager • data dictionary • system catalog • metadata • DBA • database designer • application programmer • end user

3.1 A DBMS Is Not One Program

You type eleven words:

```
SELECT name FROM student WHERE programme = 'BSIT' ORDER BY name;
```

and rows come back. Between those two events, six or seven distinct pieces of software run, each with one job. It is worth knowing them by name, because for the rest of the course every topic is really a chapter about one of them: Chapter **??** is about the optimiser and the storage manager arguing, Chapter **??** is about the transaction and recovery managers, and Chapter **??** is about the one component SQLite does not have at all.

The components of a DBMS

The **query processor** turns your statement into an answer. It contains a **parser** (is this valid SQL, and what does it refer to?), a **query optimiser** (of the many ways to compute this, which is cheapest?) and an **execution engine** (do it).

The **storage manager** stands between the query processor and the disk. It contains a **file manager** (where on disk is this?) and a **buffer manager** (which pages are in memory, and which do I evict?).

The **transaction manager** makes groups of operations behave as one, with a **lock manager** to keep concurrent users from corrupting each other's work and a **recovery manager** to put the database back together after a crash.

The **authorisation manager** decides whether this user is allowed to do this at all.

The **data dictionary** — also called the **system catalog** — is where the DBMS keeps the schema itself. It is a database about the database.

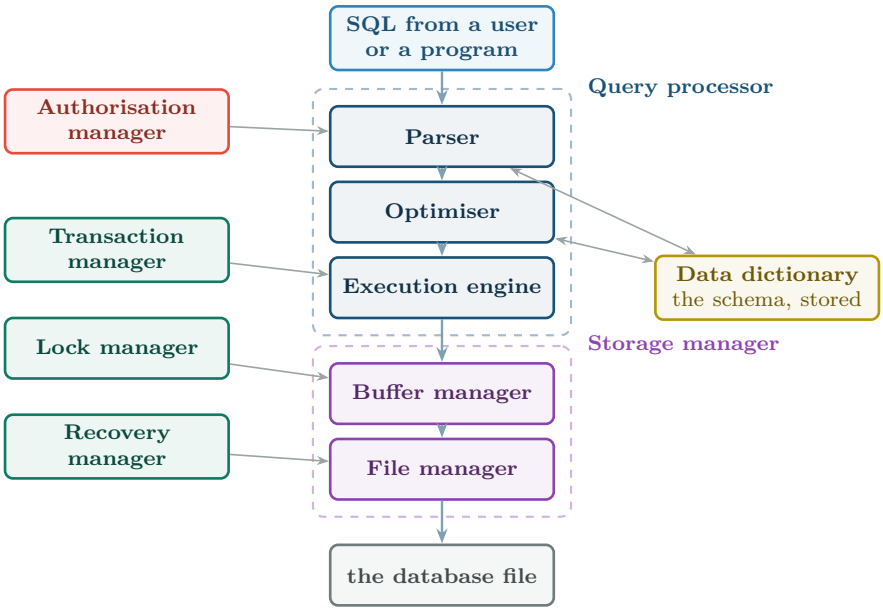


Figure 3.1: The components of a DBMS, and the path a statement takes down through them. The optimiser and the parser both consult the data dictionary — the parser to check that `student` exists and has a `name`, the optimiser to find out how big it is and what indexes it has.

3.2 Following One Statement Down

Worked Example: Eleven words, six components

```
SELECT name FROM student WHERE programme = 'BSIT' ORDER BY name;
```

1. **Authorisation manager.** May this user read `student`? In a server DBMS this is checked first and can stop everything here. *In SQLite there is no such check* — if you can open the file, you can read all of it.
2. **Parser.** Is the syntax legal? Does `student` exist? Does it have columns `name` and `programme`? Is `'BSIT'` comparable with `programme`? The answers come from the **data dictionary**, not from the data.
3. **Optimiser.** Two plans at least. Read all 4,000 rows and discard the non-BSIT ones, then sort; or, if an index on `programme` exists, use it to fetch only the BSIT rows. And if an index on `name` exists, the ORDER BY may be free — indexes are already in order. The optimiser picks by *estimated* cost, using statistics from the dictionary. Chapter **??** measures how good those estimates are.
4. **Execution engine.** Runs the chosen plan, asking the storage manager for pages as it goes.

5. **Buffer manager.** Page 7 is already in memory — no disk read. Page 112 is not, so fetch it, and evict something to make room. This is the single biggest determinant of how fast the query is, and you never see it.
6. **File manager.** Page 112 is at byte offset 111×4096 of `university.db`. Read 4,096 bytes.

The point of the exercise. You wrote *what* you wanted. Nothing in your statement said how. Steps 3 to 6 were chosen for you, and the only way to influence them is to change the schema — which is why design, and not cleverness in query writing, is most of this course.

3.3 The Data Dictionary Is Just More Tables

The claim that a DBMS stores the schema in the database is easier to believe once you have queried it.

</> The dictionary, in SQLite

Every SQLite database has a table called `sqlite_master`, and you can select from it like any other:

```
SELECT type, name, tbl_name FROM sqlite_master;
```

```
type   name          tbl_name
-----  ------------  ----------
table  student       student
index  idx_name      student
view   fees_view     student
```

The `sql` column holds the exact text of the CREATE statement that made each object — which is why `.schema` is instant: it is a SELECT, not a reconstruction.

Other systems do the same thing under other names: `information_schema` in MySQL and PostgreSQL, `ALL_TABLES` in Oracle. The idea is universal; the spelling is not.

Why this is more than a curiosity

Because the schema is data, every tool that works on data works on the schema. Three consequences you will use:

- **A program can discover a database it has never seen.** Anything that shows you a list of tables — a GUI browser, an admin page, a reporting tool — is selecting from the dictionary.
- **The schema can be backed up with the data**, because it is in the same file.
- **A dictionary query can audit a schema.** In Chapter **??** you will use one to find every table that declares a foreign key, and the point of that exercise is that the declarations are all there and none of them are being enforced.

3.4 The Four Roles

The components are software. These are people, and the distinction that matters is not job title — in a small organisation one person is all four — but *which decisions belong to which hat.*

Who decides what

The **database administrator** (DBA) owns the running system: installation, users and privileges, backups, recovery, performance, storage. The DBA decides *how it runs.*

The **database designer** owns the schema: entities, relationships, constraints, normalisation. The designer decides *what is true*.

The **application programmer** owns the programs that use the database. They decide *how it is reached*.

The **end user** owns the questions. They decide *what it is for*, and they are the only role in the list who need not know that any of the others exist.

</> One request, four reactions

"Students should be able to see their own marks on the portal, but not anybody else's."

End user	States the requirement, including the half of it that is a prohibition. Notices first when it is wrong.
Designer	Is there a reliable way to tell whose marks these are? Does ENROLMENT(roll_no, section_id, grade) tie every mark to exactly one student? If not, the requirement cannot be met at any price.
DBA	Creates the portal's database user and grants it the minimum: read on a view, and nothing on the base table.
Programmer	Writes the query with the logged-in roll number as a *parameter*, never as pasted-in text — the difference between those two is the whole of SQL injection.

Where it breaks. If the designer gets it wrong, nobody downstream can fix it: a mark not tied to one student cannot be shown to one student. This is why design comes before everything else in this course, and why Chapter **??** through Chapter **??** are four chapters in which you write almost no SQL.

⚠ In this course you are three of the four, because SQLite has no DBA

SQLite has no users, no privileges, no server to administer and no process to tune. Several things on the DBA's list simply do not exist, and the rest move onto somebody else's:

- **Users and privileges:** nobody's. The capability is absent. File permissions are the only access control there is, and Chapter **??** is about what you get back when you move to a server.
- **Backup:** yours, and it is `copy university.db` — which is genuinely all it takes, and is Chapter **??**'s subject once you ask what happens if you copy it mid-write.
- **Performance:** yours. `ANALYZE` and `EXPLAIN QUERY PLAN` are the whole toolkit, and Chapter **??** uses both.
- **Schema design, and writing the programs:** yours, which is the point — you will hold the designer's hat and the programmer's hat at the same time, and the labs are arranged so you can feel the conflict.

So: when this handout says "the DBA would", read it as "on a real system somebody else would, and this term it is you, with one hand tied".

Common Pitfalls

- **Thinking the optimiser can be instructed.** In SQL you say what, never how. If the plan is bad, change the schema or the statistics — not your tone of voice.
- **Confusing the data dictionary with the data.** The dictionary holds the schema. `SELECT COUNT(*) FROM sqlite_master` counts tables, not students.

- **Assuming the roles are four people.** They are four sets of decisions. In this course they are all you.
- **Believing "the application will check it"** is equivalent to a constraint in the schema. It is not, and Chapter **??** is eight pages on why.
- **Expecting an authorisation manager in SQLite.** There is none. Open the file and you have everything.

Try It Yourself: Query the dictionary, and watch the optimiser change its mind

```
-- --- 1. Build something for the optimiser to think about -------
sqlite3 inside.db
CREATE TABLE student (
  roll_no   TEXT PRIMARY KEY,
  name      TEXT NOT NULL,
  programme TEXT NOT NULL
);
-- 300 rows, generated by a recursive CTE (explained in Ch 15):
INSERT INTO student
WITH RECURSIVE n(i) AS (SELECT 1 UNION ALL SELECT i+1 FROM n WHERE i<300)
SELECT printf('2021-IT-%03d', i),
       printf('Student %03d', i),
       CASE i%3 WHEN 0 THEN 'BSIT' WHEN 1 THEN 'BSCS' ELSE 'BSSE' END
FROM n;

-- --- 2. Read the dictionary ------------------------------------
SELECT type, name FROM sqlite_master;
SELECT sql FROM sqlite_master WHERE name = 'student';

-- --- 3. Ask the optimiser what it intends to do ----------------
EXPLAIN QUERY PLAN
  SELECT name FROM student WHERE programme = 'BSIT' ORDER BY name;
-- Write the plan down exactly. Note the words SCAN and ORDER BY.

-- --- 4. Give it an index and ask again -------------------------
CREATE INDEX idx_prog ON student(programme);
EXPLAIN QUERY PLAN
  SELECT name FROM student WHERE programme = 'BSIT' ORDER BY name;
-- The plan changed and you did not touch the query.

-- --- 5. Make the sort free too ---------------------------------
CREATE INDEX idx_name ON student(name);
EXPLAIN QUERY PLAN SELECT name FROM student ORDER BY name;
-- Does "USE TEMP B-TREE FOR ORDER BY" still appear?

-- --- 6. Look at what you have added to the dictionary ----------
SELECT type, name, tbl_name FROM sqlite_master;
-- Three new rows you never inserted.
```

What to notice. In steps 3 to 5 the statement was identical and the plan was not. Everything that changed, changed in the schema. Keep the three plans — Chapter **??** asks you to compare them against measured times, and one of them will turn out to be a worse idea than it looks.

Checkpoint — Test Yourself

1. A query runs in 40 milliseconds on Monday and 4 seconds on Friday, on the same data and with no change to the statement or the schema. Name two components that could be responsible and say how you would tell them apart.

2. The data dictionary is stored as tables in the database it describes. Give one real advantage of that and one problem it creates.
3. For each of the four roles, state one decision that is theirs alone, and then say what happens to that decision in this course — where SQLite provides no DBA at all.

Chapter Summary

- A DBMS is several cooperating components: a **query processor** (parser, optimiser, execution engine), a **storage manager** (buffer manager, file manager), a **transaction manager** with lock and recovery managers, an **authorisation manager**, and the **data dictionary**.
- A statement travels down through them. You specify *what*; steps 3 onward — plan, pages, bytes — are chosen for you.
- The parser consults the dictionary to check names and types; the optimiser consults it for sizes and indexes.
- The **data dictionary** is ordinary tables. In SQLite it is `sqlite_master`, and `.schema` is a SELECT over it.
- Because the schema is data, programs can discover a database they have never seen, and a query can audit a schema.
- Four roles, defined by their decisions: the **DBA** decides how it runs, the **designer** decides what is true, the **programmer** decides how it is reached, the **end user** decides what it is for.
- A designer's error cannot be repaired downstream, which is why design precedes SQL in this course.
- SQLite has no authorisation manager and no DBA role. Backup, performance and design all land on you, and Chapter **??** is the chapter about what a server gives back.

Review Questions

1. (MCQ) Which component decides whether to use an index? (a) the parser; (b) the buffer manager; (c) the query optimiser; (d) the application programmer.

2. (MCQ) `sqlite_master` contains: (a) the rows of every table; (b) the schema of the database; (c) the query log; (d) the list of users.

3. (Short) Explain why the parser needs the data dictionary, with a specific example of a statement it would reject because of something it found there.

4. (Short) What does the buffer manager do, and why can it make the same query take very different times on identical data?

5. (Short) Which of the four roles is responsible when a database permits two students to have the same roll number? Justify your answer in two sentences.

6. (Applied) Take the statement `SELECT s.name, e.grade FROM student s JOIN enrolment e ON s.roll_no = e.roll_no WHERE e.grade = 'A';` and walk it through all six components, saying at each step what information that component needs and where it gets it.

7. (Applied) A college wants one database used by the examinations office, the fees office and 4,000 students. Divide the work among the four roles: list what each must do before the system opens, then mark each item that SQLite cannot support and say what you would do about it.

Chapter 4

Your First Database, in One Command

Learning Outcomes

By the end of this chapter you will be able to:

- **Create** a database, a table and a row, and **read** the row back, without installing anything.
- **Read** a relation schema written in the standard notation and **write** the CREATE TABLE that implements it.
- **State** what a database costs on disk, and **explain** why the cost goes up in jumps rather than smoothly.
- **Use** the eight dot-commands you will need every week.
- **Account for** the gap between an empty database at 0 bytes and one table at 8,192 bytes.

Before You Start

Chapter 3: the components, and the data dictionary. Appendix A for the toolkit — if `sqlite3` does not start, read that first and then come back.

Key Terms in This Chapter

CREATE TABLE • INSERT • SELECT • relation schema • column type • PRIMARY KEY • NOT NULL • dot-command • page • page size • VACUUM • database file

4.1 Three Statements and a File

Everything in Chapter 1 through Chapter 3 was description. This is the chapter where you have a database.

```
sqlite3 university.db
```

That is the installation, the server start-up, the account creation and the connection, all of it. There was no download. Then:

```
CREATE TABLE student (
  roll_no   TEXT    PRIMARY KEY,
  name      TEXT    NOT NULL,
  programme TEXT    NOT NULL,
  semester  INTEGER NOT NULL
);

INSERT INTO student VALUES ('2021-IT-001','Ayesha Khan','BSIT',2);

SELECT name FROM student WHERE roll_no = '2021-IT-001';
```

```
Ayesha Khan
```

You have now used all three levels of Chapter 2 — you wrote the conceptual schema, SQLite built the internal one, and the SELECT was an external view of one column of one row — and all six components of Chapter 3 ran. It took under a tenth of a second, and the rest of this chapter is about what that tenth of a second and that file actually cost.

4.2 Writing the Schema Down Before You Type It

The CREATE TABLE above is SQL. The thing it implements has a shorter notation, used in every textbook and in the rest of this handout:

Relation schema notation

A **relation schema** is written as the relation name followed by its attributes in brackets, with the primary key underlined:

STUDENT(roll_no, name, programme, semester)

It records the *names* and the *key* and nothing else — no types, no lengths, no storage. It is the conceptual level in one line, and it is what you write on paper while designing, before any SQL exists.

From notation to SQL, and the three things the notation loses

STUDENT(roll_no, name, programme, semester) becomes the CREATE TABLE on the previous page. Going in that direction you must *add* three decisions the notation does not carry:

- **A type for each attribute.** `roll_no` is `TEXT`, not `INTEGER`, because `2021-IT-001` is not a number — it has hyphens and leading zeros, and you will never do arithmetic on it. Choosing `INTEGER` here is the single most common first-week mistake.
- **Which attributes may be absent.** NOT NULL on three of them. Chapter **??** is about how much trouble the fourth kind — the ones that may be missing — causes.
- **The key, enforced.** The underline is a claim; PRIMARY KEY is the enforcement. Chapter **??** is about the difference between those two, and about how often the gap between them is where the bugs are.

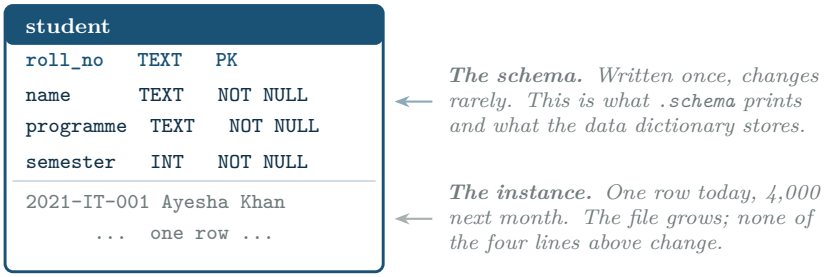


Figure 4.1: One table, with Chapter 2's schema/instance distinction drawn on it. The box above the rule is the schema and is four lines long whether the table holds one row or a million.

4.3 What It Actually Cost

A claim that a tool is lightweight is worth nothing unless somebody weighs it. The harness `code/m04_firstdatabase.py` does the three statements above and measures them.

Measured: From nothing to a row, and what the file weighs

SQLite 3.51.2, Python 3.13.13, Intel i5-6600 (4 cores, 3.30 GHz), 16 GB RAM, Windows 11, mechanical disk.

1. Time from no database at all to a row read back

`connect`	0.52 ms	the file does not exist yet
`create`	25.27 ms	one CREATE TABLE
`insert`	23.80 ms	one row
`select`	0.16 ms	read it back
TOTAL	**49.75 ms**	→ `'Ayesha Khan'`

No server was started, no port opened, no password set, nothing installed.

2. Bytes on disk

an empty database, just connected	0
after one CREATE TABLE, no rows	8,192
1 row	8,192
100 rows	8,192
1,000 rows	32,768
10,000 rows	262,144
the same 10,000 rows after VACUUM	262,144

3. Where the growth happens (page size 4,096 bytes)

at row	bytes	pages	bytes/row so far
1	8,192	2	8192
181	16,384	4	91
351	20,480	5	58

Read the three results, in order.

- **An empty database is 0 bytes.** Connecting does not write anything. The file appears when the first statement that changes something commits — which is already Chapter **??**'s subject showing through.
- **The first table costs 8,192 bytes, and so does the hundredth row.** Two pages: one for SQLite's own header and `sqlite_master` — the data dictionary from Chapter 3, which has to live somewhere — and one for your table. That second page then holds about 180 of these rows for nothing.
- **The file grows in jumps of exactly 4,096 bytes**, at rows 1, 181 and 351. It does not creep up per row. The cost of storage is quantised, and the unit is the page.
- VACUUM **recovered nothing**, because nothing had been deleted. It is not a compressor; it reclaims space that deletions freed. Students run it hopefully and it is worth knowing now that hope is not the mechanism.

On re-running this. The total time varies: repeated runs on this machine gave 49.75 ms and 70.38 ms. Nothing changed but the operating system's disk cache — which is the buffer manager's neighbour from Chapter 3 making itself felt in week four. **The bytes do not vary at all.** When you report a measurement, say which kind you have: the timings are a range, the page counts are a fact.

Why 8,192 bytes for an empty table is the right answer

It looks like waste — 8 KB to store nothing. It is the price of being a database rather than a file, and it buys exactly two things:

- **The dictionary.** Page 1 holds the schema. Because of it, any program can open `university.db` and discover what is in it. The three CSV files in Chapter 1 had no such page, which is precisely why a program could not tell what they meant.

- **Room to grow in place.** Page 2 is mostly empty on purpose. Rows 2 through 180 cost nothing because the space is already there, and this is why inserting is fast.

At 10,000 rows the overhead has stopped mattering: 262,144 bytes is about 26 bytes a row, and the rows themselves are about that size. **The fixed cost is paid once and the marginal cost is the data.** That is the shape you want from a storage system, and Chapter **??** is the chapter where you deliberately pay the fixed cost a second time to make reads fast.

⚠ The comparison that makes "lightweight" mean something

"Lightweight" is only a claim about something else. Counting what each arrangement requires before the first row exists:

	SQLite	**a server**	
installation steps	0	several	download, install, configure
background processes	0	≥ 1	a service that must be running
network ports open	0	≥ 1	3306, 5432, 1433 …
accounts to create	0	≥ 1	starting with the superuser
config files to edit	0	≥ 1	before anything is tuned
things that can be **misconfigured**	**0**	**all of the above**	

Every zero in the left column is a thing that cannot go wrong in week one — and also **a thing you will not learn to do this term**. That is the trade, stated plainly: you get sixteen weeks on data and none on administration. Chapter **??** is where the right-hand column is taught by contrast, and where you write the GRANT statements that SQLite will not run.

4.4 The Eight Dot-Commands

Dot-commands are instructions to the `sqlite3` shell, not SQL. They take no semicolon. There are about sixty; you will use these eight all term.

</> The working set

`.open university.db`	open a database from inside the shell
`.tables`	list the tables
`.schema student`	print the CREATE TABLE text, exactly as stored
`.headers on`	put column names above the output — **set this first, always**
`.mode box`	draw results as a table instead of pipe-separated text
`.read v1_create.sql`	run a file of SQL
`.once out.txt`	send the next result to a file
`.quit`	leave

`.headers on` and `.mode box` together are the difference between output you can read and output you cannot. Appendix A has the full list and the two-line reset procedure for when a lab database gets into a state you do not want to debug.

Common Pitfalls

- **A semicolon after a dot-command.** `.tables;` is an error. SQL takes a semicolon; dot-commands do not.
- **Forgetting the semicolon after SQL.** The shell prints `...>` and waits. It is not frozen — it wants the rest of your statement.
- **Making `roll_no` an `INTEGER`.** `2021-IT-001` is text. This one costs a schema rewrite in week six.
- **Expecting VACUUM to shrink a database.** It reclaims deleted space only. With no deletions it recovers nothing, as measured above.
- **Running `sqlite3` with no filename.** You get a database in memory, you can work in it happily for an hour, and it vanishes on `.quit` with no warning.
- **Judging a database's size by the file alone.** 8,192 bytes held 100 rows and also 1. The file size tells you how many pages were allocated, not how much data there is.

Try It Yourself: Build the first table of the database you will keep all term

This is the beginning of the semester project. Keep `university.db` — every lab from here adds to this one file, and Chapter **??** submits it.

```
# --- 1. Create it, in the folder you will keep all term --------------
sqlite3 university.db
```

```
-- --- 2. Make the output readable before anything else -----------
.headers on
.mode box

-- --- 3. The first table -----------------------------------------
CREATE TABLE student (
  roll_no   TEXT    PRIMARY KEY,
  name      TEXT    NOT NULL,
  programme TEXT    NOT NULL,
  semester  INTEGER NOT NULL
);

-- --- 4. Four rows -----------------------------------------------
INSERT INTO student VALUES
  ('2021-IT-001','Ayesha Khan','BSIT',2),
  ('2021-IT-002','Bilal Ahmed','BSIT',2),
  ('2021-CS-007','Hira Nadeem','BSCS',4),
  ('2021-SE-013','Usman Tariq','BSSE',2);

-- --- 5. Read them back ------------------------------------------
SELECT * FROM student;
SELECT name, programme FROM student WHERE semester = 2;

-- --- 6. Prove the key is enforced -------------------------------
INSERT INTO student VALUES ('2021-IT-001','Someone Else','BSIT',2);
-- Write down the exact error message. This is Chapter 8 arriving early.

-- --- 7. Leave the shell -----------------------------------------
.quit
```

```
# --- 8. Measure your own machine ---------------------------------
# the file size, and the pages it is made of
ls -l university.db
sqlite3 university.db "PRAGMA page_size; PRAGMA page_count;"
python code/m04_firstdatabase.py
```

Record and keep. Your own total milliseconds, your own page size, and your own three growth points. They will not match the measured box exactly — different disk, different cache — and **the ones that should match exactly are the byte counts**. If your page size is not 4,096, say so when you hand in, because every size figure in this handout scales by it.

Checkpoint — Test Yourself

1. An empty database is 0 bytes and one empty table is 8,192. Account for every one of those bytes — what is in them, and why could it not be fewer?
2. The file was 8,192 bytes at 100 rows and 32,768 at 1,000. Predict the size at 2,000 rows, show the arithmetic, then run it and explain any difference between your prediction and the result.
3. Write INSTRUCTOR(emp_id, name, department, designation) as a CREATE TABLE, choosing a type for each attribute, deciding which may be NULL, and justifying both choices for `emp_id` in one sentence each.

Chapter Summary

- `sqlite3 university.db` is the whole of installation, start-up and connection. Three statements — CREATE TABLE, INSERT, SELECT — are a working database.
- A **relation schema** is written STUDENT(roll_no, name, programme, semester): name, attributes, key underlined. Turning it into SQL means adding types, nullability and real key enforcement.
- Measured: **49.75 ms** from nothing to a row read back; an **empty database is 0 bytes**; one CREATE TABLE costs **8,192 bytes** and then holds about 180 rows free; 10,000 rows is **262,144 bytes**, about 26 bytes a row.
- The file grows in **4,096-byte page jumps**, at rows 1, 181 and 351 — not smoothly. Storage is quantised and the unit is the page.
- The 8 KB fixed cost buys the data dictionary and room to grow in place. It is paid once; after that the marginal cost is the data.
- VACUUM reclaims space freed by deletions. With no deletions it recovers nothing.
- Timings vary with the OS disk cache (49.75 vs 70.38 ms on the same machine); byte counts do not vary. Report which kind of number you have.
- "Lightweight" means zero installation steps, processes, ports, accounts and config files — and therefore zero of those to misconfigure, and zero of them learned this term.
- Eight dot-commands cover the course. Set `.headers on` and `.mode box` first, every session.

Review Questions

1. (MCQ) A newly connected, empty SQLite database occupies: (a) 0 bytes; (b) 4,096 bytes; (c) 8,192 bytes; (d) one page per table.

2. (MCQ) VACUUM on a database from which nothing has been deleted: (a) halves the file; (b) compresses the rows; (c) recovers nothing; (d) rebuilds the indexes only.

3. (Short) Why is `roll_no` declared `TEXT` and not `INTEGER`? Give two reasons, one about correctness and one about what you would eventually want to do with the value.

4. (Short) Explain why the file size jumped at row 181 and not at row 150 or row 200.

5. **(Short)** The same harness gave 49.75 ms and 70.38 ms on the same machine with the same data. Is one of them wrong? Answer in two sentences, saying what you would report.

6. **(Applied)** Write the CREATE TABLE for COURSE(course_code, title, credit_hours, department), then predict its empty size in bytes and the row count at which it will first grow. Run it, record both, and explain any difference between prediction and measurement.

7. **(Applied)** A colleague proposes storing the university's data in one table of 40 columns "to avoid joins". Using the page measurements in this chapter, estimate what that costs in bytes at 10,000 students compared with the four-column table above, and then give the argument against it that is *not* about bytes — Chapter **??** will measure both halves of this answer.

Appendices

Chapter A

The Toolkit: Setting It Up and Keeping It

This appendix is the whole of the course's setup. It is short, and that is the point: the longest-running complaint about database courses is that the first fortnight goes on installing something. Here there is nothing to install.

A.1 What You Need

Thing	Version used here	How to get it
SQLite	3.51.2	**You already have it.** It ships inside Python, inside most operating systems, and inside a great deal of software you already run
A terminal	any	Already present
Python	3.13	Only for the measurement harnesses in `code/`. No chapter's *content* needs it
A text editor	any	For writing `.sql` files

Check, in one line, before you read any further

```
python -c "import sqlite3; print(sqlite3.sqlite_version)"
```

If that prints a version number — any version from 3.37 onwards — you have everything this course needs. If you also have the `sqlite3` command on your path, so much the better; if not, every lab works through Python's built-in shell instead, and Chapter 4 shows both.

A.2 Why SQLite, and What It Costs You

This is a curriculum decision, not a shortcut

HEC names no database product in any edition of this curriculum. The 2017 and 2023 outlines say a student should use SQL "in any DBMS"; the 2025 outcomes say "popular database management systems". There is no edition in which choosing SQLite departs from the published syllabus.

What it buys is the first fortnight. There is no server to start, no port, no password, no user account, no configuration file and nothing to go wrong before the subject begins. A database is one file. You can copy it, email it, put it in version control, delete it and start again — and in Chapter **??** you will do exactly that.

What it costs is stated here and again in Chapter ??, because a tool's limits are part of what you are learning:

- **No users, no GRANT, no REVOKE.** SQLite has no access control of any kind: whoever can open the file can do anything to it. Chapter **??** writes the SQL a server DBMS would use and runs it nowhere.
- **No stored procedures.** Triggers (Chapter **??**) carry the argument about putting logic in the database.

- **No row-level locking and no isolation levels.** SQLite lets one writer at a time and makes the others wait. Chapter **??** can therefore show you a lost update prevented, a busy timeout and a failed upgrade from a deferred transaction — but a genuine two-row deadlock is drawn rather than run.

Every one of those three is a thing a *server* adds, and knowing which three they are is more useful than having had them switched on all semester without noticing.

A.3 The Ten Commands You Will Actually Use

SQLite's shell has two kinds of input: SQL, which ends with a semicolon, and *dot-commands*, which do not and which are the shell's own.

Command	What it does
`sqlite3 university.db`	Open (or create) a database and enter the shell
`.tables`	List the tables
`.schema student`	Show the `CREATE` statement for one table — the fastest way to find out what a table actually promises
`.headers on`	Print column names above results. Turn this on first, every time
`.mode box`	Draw results as a table instead of pipe-separated text
`.read setup.sql`	Run a file of SQL. How every lab in this book starts
`.output results.txt`	Send what follows to a file
`.import data.csv student`	Load a CSV into a table
`.backup backup.db`	Take a consistent copy *while the database is open* — Chapter **??** explains why that is not the same as copying the file
`.quit`	Leave

</> The two lines to type at the start of every session

```
sqlite3 university.db
```

then, inside the shell:

```
.headers on
.mode box
PRAGMA foreign_keys = ON;
```

The first two make results readable. **The third is not cosmetic.** SQLite does not enforce foreign keys unless you ask it to, *every time you connect*, and Chapter **??** is largely about what happens when you forget. Put all three in a file called `init.sql` and start every session with `.read init.sql` until it is a habit.

A.4 Resetting to a Known State

A database is one file, which makes this the simplest reset in the series:

```
# --- 1. Keep a copy before anything risky ------------------------
cp university.db university.db.bak

# --- 2. Start again from nothing ---------------------------------
rm university.db
sqlite3 university.db ".read code/schema/v1_create.sql"

# --- 3. And refill it --------------------------------------------
python code/schema/seed.py 1000
```

Do this deliberately once, in Week 2

Delete your database on purpose, rebuild it from the script, and confirm it is the same. A student who has done that once is a student who will keep their schema in a file rather than in the shell's history — which is the single habit that decides whether the Week 16 project can be handed to somebody else and run.

A.5 The Machine Every Timing Was Taken On

Eleven chapters quote a measured figure. Every one was produced by running the named file in `code/` on this machine:

Component	Specification
Processor	Intel Core i5-6600, 4 cores, 3.30 GHz
Memory	16 GB
Operating system	Windows 11, 64-bit
SQLite	3.51.2
Python	3.13.13
Storage	Mechanical disk, not a solid-state drive — which matters for Chapter **??** and is the reason its figures are larger than yours may be

Your numbers will differ, and the ratios should not

A measurement is a fact about the machine it ran on. If your laptop has a solid-state disk it will beat these figures, perhaps by a factor of ten on the chapters that touch storage.

What should survive the move is the shape: the row count at which an index starts to pay (Chapter **??**), the direction normalisation moves query time (Chapter **??**), the fact that one writer blocks another (Chapter **??**). If a *ratio* in this book does not reproduce on your machine, that is worth investigating and worth telling your lecturer about. If an absolute number does not, that is expected.

A.6 Scope: What You May Point Any of This At

Read this before Chapter 23's laboratory

Everything in this course runs against **a database file you created, on a machine you own**. Chapter **??** discusses SQL injection and shows what a badly built query permits — against your own university database, with invented data, in a file you can delete.

Running the same thing against any system you do not own and have not been authorised in writing to test is a criminal offence in Pakistan under the **Prevention of Electronic Crimes Act 2016**, sections 3 and 4, which cover unauthorised access to an information system and to data.

There is no educational exemption, and a university assignment is not authorisation. "I was learning", "I did no harm" and "the site was insecure" are not defences.

In practice this is an easy rule to follow, because the only file named anywhere in this book is `university.db` and it is yours.

Chapter B

The Sixteen-Week Schedule

This appendix is the **source of record** for the teaching schedule. The one-page `DBS_Fall2026_Lecture_Schedule.pdf` distributed to students is generated from it, and the slide pipeline parses the table below. Change this first; regenerate the one-page version afterwards.

Twenty-three chapters are taught across sixteen weeks, so seven weeks carry two chapters. The pairings are not arbitrary: in every case the two are either one topic the printed book splits for length — the two halves of the ER model, the two halves of decomposition — or a pair in which the second is unusable without the first.

Two weeks are deliberately left alone. **Chapter 18, Normal Forms, has a week to itself**, although it would pair naturally with Chapter 17. Its laboratory breaks the running database and rebuilds it, and that does not fit in half a week alongside anything else. **Chapter 21, Indexes**, likewise: the measurement at a million rows takes most of a laboratory session to run and the rest of it to argue about.

B.1 The Schedule

Wk	Lecture topic	Chapters	Laboratory	Assessment
1	Why not files; the three-level architecture	1, 2	Four questions a file cannot answer, asked of a file	—
2	Who does what; your first database	3, 4	Install nothing; create a database and measure what it cost	—
3	The entity-relationship model	5	Entities and attributes for the university, on paper	—
4	Cardinality, participation, weak entities	6	The university ER diagram, finished and argued over	—
5	The relational model; keys and constraints	7, 8	Orphan rows in a correctly declared schema	**Quiz 1 (Ch 1–6)**
6	Mapping a diagram to tables	9	Your diagram becomes eleven tables, by rule	**Assignment 1 set**
7	Creating and changing a schema	10, 11	`CREATE TABLE`; then change one that is already full	—
8	Relational algebra	12	Midterm, then algebra by hand on small relations	**Midterm (Ch 1–11)**
9	Joins; selecting rows and NULL	13, 14	Every join form on the same two tables, and what NULL does to each	**Assignment 1 due**

Wk	Lecture topic	Chapters	Laboratory	Assessment
10	Sub-queries; grouping and aggregation	15, 16	Twenty questions, each answerable two ways	**Project proposal**
11	Functional dependencies	17	Closures and a minimal cover, by hand then checked	—
12	Normal forms: 1NF to BCNF	18	Break the running database, then normalise it and measure both	**Quiz 2 (Ch 12–17)**
13	Decomposition; views and triggers	19, 20	A lossy decomposition you can see losing data	**Assignment 2 set**
14	Indexes, and where they stop helping	21	One million rows, with and without an index	—
15	Transactions, concurrency and recovery	22	Two writers, one database, and a lost update	**Assignment 2 due**
16	Security, NoSQL and the project	23	Project presentations	**Project due; presentations**

B.2 How the Assessment Adds Up

HEC publishes the assessment instruments for this course — sessional examination, home assignments, quizzes, project, presentations, final examination — without weights. The weights below are the department's, and are stated here so that a student can see the whole shape of the semester on one page.

Instrument	Weight	What it examines
Two quizzes	10%	Weeks 5 and 12. Short, closed-book, on the chapters named in the schedule
Two assignments	10%	Set in weeks 6 and 13. Both are done on your own database and submitted as a change to it plus a short write-up
Laboratory	**15%**	Assessed weekly from Week 2. This course carries a **credited laboratory** — one of its four credit hours — and the marks follow the credit
Midterm examination	20%	Week 8, Chapters 1–11
Semester project	20%	The database itself, built across the whole semester (Chapter **??**), submitted in week 16 with its presentation
Final examination	25%	All chapters, with emphasis after the midterm

⚠ The laboratory is a quarter of the course, not a formality

This is the only course in the series with a *credited* laboratory: HEC specifies 4 credit hours as 3+1 in 2017 and 2025, and as 4 (3–3) in 2023. One of those four hours is the lab, and 15% of the marks follow it.

That is also why the laboratories in this handout are longer than those in its sibling courses. They are not homework to be fitted around the lectures; they are a timetabled hour with a demonstrator in the room, and every one of them adds something to the database you will submit in Week 16.

Chapter **??** states what is required and how it is marked. Read it in Week 3, not in Week 15.